

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

CE-403: Database Administration (PostgreSQL)

Elchin Salahzada

Lecture 4

CE-403: Database Management Systems
Baku Higher Oil School : Computer Engineering

Aggregate Functions & Grouping

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)
- Grouping Data with GROUP BY
- Filtering Groups with HAVING
- Window Functions – Introduction & OVER()
- Ranking Functions (ROW_NUMBER, RANK, DENSE_RANK)
- PARTITION BY and Frame Clauses
- Lab Activity

Learning objectives

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Apply aggregate functions (COUNT, SUM, AVG, MIN, MAX) to summarize data
- Group data using GROUP BY and filter grouped results using HAVING
- Explain the difference between WHERE and HAVING, and between GROUP BY and window functions
- Write window functions using the OVER() clause to perform calculations across a set of rows without collapsing them
- Apply ranking functions (ROW_NUMBER, RANK, DENSE_RANK) to order and rank data within partitions
- Use PARTITION BY and frame clauses to control how window functions calculate results

Aggregate Functions

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Aggregate functions perform a calculation on a set of rows and return a single value
- COUNT() – counts the number of rows
  - SELECT COUNT(*) FROM employees;
- SUM() – adds up numeric values
  - SELECT SUM(salary) FROM employees;
- AVG() – calculates the average
  - SELECT AVG(salary) FROM employees;
- MIN() / MAX() – finds the smallest/largest value
  - SELECT MIN(salary), MAX(salary) FROM employees;
- Aggregate functions ignore NULL values (except COUNT(*))
- Often used with GROUP BY to summarize data per category

Grouping Data with GROUP BY

- GROUP BY groups rows that share the same values in specified columns, allowing aggregate functions to be applied per group
- Basic syntax:
  - SELECT column, aggregate_function(column)
  - FROM table_name
  - GROUP BY column;
- Example: Count employees per department
  - SELECT dept_id, COUNT(*) AS total_employees
  - FROM employees
  - GROUP BY dept_id;
- Every column in SELECT that is not an aggregate function must appear in GROUP BY
- Can group by multiple columns for more granular grouping

Filtering Groups with HAVING

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- HAVING filters groups created by GROUP BY, based on a condition applied to aggregate results
- Basic syntax:
  - SELECT column, aggregate_function(column)
  - FROM table_name
  - GROUP BY column
  - HAVING condition;
- Example: Departments with more than 5 employees
  - SELECT dept_id, COUNT(*) AS total_employees
  - FROM employees
  - GROUP BY dept_id
  - HAVING COUNT(*) > 5;

WHERE vs. HAVING

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- WHERE filters individual rows before grouping
- HAVING filters groups after aggregation
- Can combine both:
    - SELECT dept_id, AVG(salary)
    - FROM employees
    - WHERE age > 25
    - GROUP BY dept_id
    - HAVING AVG(salary) > 50000;

Window Functions

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Window functions perform calculations across a set of rows related to the current row — without collapsing rows like GROUP BY
- Defined using the OVER() clause
    - SELECT column,
    - aggregate_function(column) OVER (...)
    - FROM table_name;
- Example: Show each employee's salary alongside the department average (without grouping)
    - SELECT name, dept_id, salary,
    - AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg_salary
    - FROM employees;
- Unlike GROUP BY, window functions keep every row in the result
- Common use cases: running totals, rankings, moving averages, comparisons within groups
- Any aggregate function (SUM, AVG, COUNT, etc.) can be used as a window function with OVER()

Ranking Functions

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Ranking functions assign a rank/number to each row, typically used with ORDER BY inside OVER()
- ROW_NUMBER() – assigns a unique sequential number to each row
  - SELECT name, salary,
  - ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num
  - FROM employees;
- RANK() – assigns rank, skips numbers after ties
  - SELECT name, salary,
  - RANK() OVER (ORDER BY salary DESC) AS rank
  - FROM employees;
- DENSE_RANK() – assigns rank, no gaps after ties
  - SELECT name, salary,
  - DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank
  - FROM employees;
- Example difference: For tied values, RANK may jump (1, 1, 3), while DENSE_RANK stays sequential (1, 1, 2)

PARTITION BY and Frame Clauses

- PARTITION BY divides rows into groups (partitions) — the window function is calculated separately within each partition
    - SELECT column,
    - function() OVER (PARTITION BY column ORDER BY column)
    - FROM table_name;
- Example: Rank employees by salary within each department
    - SELECT name, dept_id, salary,
    - RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dept_rank
    - FROM employees;
- Frame Clause – defines which rows within the partition are included in the calculation (ROWS BETWEEN ... AND ...)

Example:

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Example: Running total of salaries ordered by hire date
  - SELECT name, hire_date, salary,
  - SUM(salary) OVER (ORDER BY hire_date
  - ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
  - FROM employees;
- Common frame options: UNBOUNDED PRECEDING, CURRENT ROW, UNBOUNDED FOLLOWING

Lecture Summary

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Aggregate functions (COUNT, SUM, AVG, MIN, MAX) summarize data into a single value
- GROUP BY groups rows for per-category aggregation; HAVING filters those groups after aggregation
- Window functions perform calculations across related rows without collapsing them, using OVER()
- Ranking functions (ROW_NUMBER, RANK, DENSE_RANK) assign order/rank within result sets
- PARTITION BY splits data into groups for window calculations; frame clauses control which rows are included
- Window functions enable running totals, moving averages, and rankings — impossible with GROUP BY alone

Hands-on

BAKI ALİ NEFT MƏKTƏBİ
BAKU HIGHER OIL SCHOOL

- Scenario: Analyze employees data using aggregation and window functions
- Count total employees and average salary per department (GROUP BY)
- Find departments with more than 3 employees (HAVING)
- Show each employee's salary alongside their department's average salary (OVER (PARTITION BY))
- Rank employees by salary company-wide (RANK, DENSE_RANK, ROW_NUMBER)
- Rank employees by salary within each department (PARTITION BY + ORDER BY)
- Calculate a running total of salaries ordered by hire_date (frame clause)
- Compare WHERE vs HAVING using one combined query (filter rows, then filter groups)

Next Lecture

- Database Design & ER Modeling